

ReconPilot

Every concept behind a production-shaped fintech platform
Spring Boot · PostgreSQL · Docker · React · System Design

A complete reference for the UPI MDR reconciliation platform: what each technology does, why each decision was made, every bug we hit and what it taught, and a step-by-step guide to rebuilding the whole thing from nothing.

Java 21

Spring Boot 4.1

PostgreSQL 17

Docker / Colima

React 19 + TypeScript

Redux Toolkit

Testcontainers

87 tests

Written for Vaibhav Jadhao · Built September 2026

Every number, error message and measurement in this document came from the real system.

How to use this document

This is not a tutorial to follow blindly. It is the reasoning behind a system you already built, written so you could rebuild it alone — and so you can defend every decision in an interview.

The three questions every section answers

QUESTION	WHY IT MATTERS
What is it?	A plain-language explanation with a real-world analogy before any jargon.
Why did we use it?	The specific problem in ReconPilot it solved. A technology with no problem attached is trivia.
What did it cost?	Every choice gives something up. Knowing the tradeoff is the difference between using a tool and understanding it.

THE ONE HABIT WORTH TAKING FROM ALL OF THIS

Roughly a third of this document is bugs — not because the work was sloppy, but because **every one of them was found by testing a belief rather than trusting it**. Row-level security that was configured and did nothing. A test suite that destroyed a development database. A rounding difference of one paisa that produced 8,174 false claims.

None of those would have been found by reading the code. All of them were found by asking "*is this actually true?*" and then checking. That habit is worth more than any framework on the cover.

What ReconPilot does

From 15 October 2026, India's UPI charges merchants a fee (MDR) on payments above ₹2,000 — ending six years of zero-cost UPI. The rules are conditional and stateful, and they are being implemented in weeks by dozens of banks and payment providers whose billing systems were all built assuming the fee was zero.

They will get it wrong. ReconPilot independently recomputes what each merchant *should* have been charged, compares it to what they *were* charged, and drives the difference to recovery.

```
Settlement file (2 GB)
  ↓ streamed, never loaded whole
Event log           1,000,000 transactions in 27 seconds, 256 MB heap
  ↓ recompute every row against the published rules
Breaks              7,300 discrepancies · ₹133,846 recoverable · 3.0 seconds
  ↓ claim, file, chase
Recovered money      audited, guarded, one state machine
```

Contents

1. **The machine** — Homebrew, the JDK, Maven, Node, Colima
2. **Docker** — containers, images, volumes, Compose, and why macOS runs Linux
3. **PostgreSQL** — schema design, money, time, indexes, and row-level security
4. **Spring Boot** — dependency injection, proxies, auto-configuration, security
5. **Testing** — unit vs integration, Testcontainers, and tests that can actually fail
6. **React & TypeScript** — components, Redux Toolkit, RTK Query, MUI
7. **System design** — the ideas that transfer to any language
8. **Every bug, and what it taught**
9. **Rebuild it from nothing** — step by step
10. **Interview answers** — the short versions

1 • The machine

Before any code, the tools. Each one exists because a specific job would otherwise be done by hand, badly, repeatedly.

1.1 Homebrew — a package manager for your operating system

ANALOGY

When you want WhatsApp on your phone, you don't visit their website, work out which build matches your chip, download an installer and tell the phone where it went. You open the Play Store and tap install. The store finds it, fetches the right version, installs it, and updates it later.

macOS has no such store for developer tools. Homebrew is the one the community built.

Installing Java *without* Homebrew means: pick a vendor (Oracle? Adoptium? Amazon? Azul?), pick a build (ARM64 or x86? JDK or JRE? 17, 21 or 25?), download a `.dmg`, run an installer, set `JAVA_HOME` by hand, add it to `PATH` by hand — and repeat all of it to upgrade or to run two versions side by side.

With Homebrew: `brew install openjdk@21`.

THE PATTERN TO NOTICE

Homebrew is a **dependency manager** — the same idea as Maven for Java or npm for Node, operating one layer down at the level of your operating system. You declare what you want; it works out what *that* needs, fetches everything, and records it so it can be removed cleanly.

"**Declare intent, let the tool resolve the graph.**" You will meet this idea four more times in this document: in Maven, in Docker, in Docker Compose, and in Spring's dependency injection. Recognising a pattern across technologies is most of what makes an experienced engineer fast in an unfamiliar stack.

1.2 The JDK, and why version choice is an engineering decision

Java ships a new version every six months, in March and September. Most are **feature releases**, supported for about six months. Roughly every two years one is designated **LTS** — **Long Term Support** — and receives security patches for years.

LTS line: 8 → 11 → 17 → 21 → 25

Enterprises run LTS versions, almost without exception. No bank runs Java 27 in production.

	JAVA 21	JAVA 25
Released	September 2023	September 2025
Enterprise adoption in 2026	Dominant	Early adopters
What Indian JDs ask for	17 or 21	rarely
Library compatibility	Everything works	Occasional rough edges

We chose **21**. You are learning Java to be hired, so you want fluency in what companies actually run. Java 21 is also where **virtual threads** landed, which is the hottest Java interview topic right now. Nothing is lost: everything you learn on 21 works identically on 25.

OpenJDK vs "distributions" — the thing that confuses everyone

OpenJDK is the source code — the free, open-source reference implementation. Nobody downloads "OpenJDK the product"; you download someone's *build* of it: Eclipse Temurin, Amazon Corretto, Azul Zulu, Microsoft's build, Oracle JDK.

ANALOGY

OpenJDK is the recipe. Temurin, Corretto and Zulu are different bakeries using it. The bread is essentially identical — all are TCK-certified, meaning they pass Oracle's official compatibility suite. They differ in support contracts, release cadence and minor tuning.

This matters practically: Oracle's own JDK has had licensing terms that made companies nervous about production use, which is precisely why Temurin and Corretto exist. If asked "*which JDK and why?*", answering "*Temurin, because it is TCK-certified and free of Oracle's commercial licensing constraints*" is a strong answer for a two-year developer.

JAVA_HOME — what it is and why builds break without it

An **environment variable** is a note pinned to your shell that every program can read. `JAVA_HOME` is the note saying "*the JDK lives at this exact path.*" Maven, Gradle, IntelliJ, Tomcat and nearly every Java tool read it rather than guessing.

```
# in ~/.zprofile
export JAVA_HOME="/opt/homebrew/opt/openjdk@21/libexec/openjdk.jdk/Contents/Home"
export PATH="$JAVA_HOME/bin:$PATH"
```

THIS BIT US TWICE

`~/.zprofile` is read only by **zsh login shells**. IntelliJ launches Maven through `/bin/sh`, so `JAVA_HOME` never reached it and it fell back to JDK 27 — where Lombok's annotation processor crashed with `ExceptionInInitializerError: com.sun.tools.javac.tree.EndPosTable`.

Configuration that lives in a shell profile is invisible to anything that is not that shell. Not `/bin/sh`, not an app launched from the Dock, not a CI runner, not a Docker container. A build that only works because of your shell is not reproducible — which is exactly what CI exists to catch.

1.3 Maven — dependencies plus a standard build

You already know this concept from two directions: `package.json` and `npm` in Angular, NuGet and MSBuild in .NET. Maven is both, plus strong opinions about your folder layout.

Three jobs

1. Dependency management. You write in `pom.xml`: *"I need Spring Boot Web."* Maven downloads it — and everything it needs, and everything *those* need, recursively. That recursion is called **transitive dependencies** and is the whole reason these tools exist. Spring Boot Web pulls in around thirty libraries you never asked for and shouldn't have to think about.

WE WATCHED THIS HAPPEN LIVE

`brew install maven` printed:

```
==> Installing maven dependency: openjdk
📦 /opt/homebrew/Cellar/openjdk/27: 535 files, 391.2MB
```

Maven dragged Java 27 in as its own dependency. Suddenly there were four JDKs on the machine — and `JAVA_HOME` is what decided which one Maven actually used.

2. A standard build lifecycle.

`validate` → `compile` → `test` → `package` → `verify` → `install` → `deploy`

`mvn package` runs every phase up to and including `package`. The point is that **every Maven project on earth builds the same way** — you can clone a project you have never seen and know that `mvn package` will build it.

3. Convention over configuration. Maven doesn't ask where your source is. It knows:

<code>src/main/java</code>	your code
<code>src/main/resources</code>	config files
<code>src/test/java</code>	your tests
<code>target/</code>	build output

BURN THIS PHRASE INTO MEMORY

"**Convention over configuration**" means *the tool assumes sensible defaults so you only configure the exceptions*. Spring Boot is built entirely on this philosophy. When an interviewer asks "*why is Spring Boot better than plain Spring?*", this idea is the answer.

The Maven wrapper — always use `./mvnw`

`mvnw` is a script committed into the project that downloads the *correct* Maven version automatically. It is why anyone can clone the repo and build without installing Maven first. `./mvnw` beats `mvn` every time.

The `./` is not decoration: it tells the shell to look *in this exact directory* rather than searching `PATH`. That is why it only works from the folder containing it.

1.4 Node and npm

Same LTS reasoning as Java. Node 26 was the newest release; **Node 24 was the current Active LTS**, so we used 24. npm is the package manager; `package.json` is the manifest; `node_modules` is where the resolved graph lands (and is never committed).

1.5 Colima instead of Docker Desktop

Docker Desktop failed to install: it needed `sudo` to create `/usr/local/bin`. Rather than force it, we used **Colima** — and it turned out to be the better choice anyway.

	DOCKER DESKTOP	COLIMA
Install	needs your password	no password
RAM overhead	heavy (~2–4 GB idle)	light, and you set the cap
GUI	yes	no — CLI only
Commands	<code>docker</code> , <code>docker compose</code>	identical
Commercial licence	required for companies >250 staff	free, Apache 2.0

A LICENSING POINT WORTH KNOWING AT BARCLAYS

Docker Desktop requires a **paid commercial licence** for organisations with over 250 employees or \$10M+ revenue. Barclays is comfortably over both. Many engineers install it on work machines without knowing this. Colima is Apache 2.0 with no such restriction. Knowing where licensing boundaries sit is a genuinely senior habit.

```
colima start --cpu 4 --memory 6 --disk 60 # start the Linux VM
colima status                             # check
colima stop                               # reclaim the RAM
```

REMEMBER THIS ONE

Colima does not start automatically at boot. If `docker` ever fails with "*cannot connect to the Docker daemon*", that is almost always why.

2 • Docker

The single most important infrastructure concept in the project, and the one that transfers to every job you will ever have.

2.1 The problem it solves

Every developer has said this sentence: "*But it works on my machine.*" The code runs locally and breaks on the server — different OS, different library version, different config. Hours die here.

ANALOGY — THE SHIPPING CONTAINER

Before the 1950s, cargo ships were loaded with loose goods: barrels, sacks, crates, all different shapes. Every port needed different equipment, every transfer was slow and manual, and loading a ship took days.

Then came the **standard shipping container**. Suddenly it didn't matter whether it held bananas or televisions — every container was the same size with the same fittings, so every crane, ship, truck and train handled it identically. This single boring invention is credited with reshaping global trade.

Docker is that container, for software. You package your app *plus* its exact dependencies *plus* its config into one standard unit. Every machine that can run containers runs yours identically. The host does not need to know or care what is inside.

2.2 Container vs virtual machine

You will be asked this in interviews.

	VIRTUAL MACHINE	CONTAINER
Ships	an entire operating system	your app and its libraries
Size	gigabytes	megabytes
Start time	tens of seconds	about a second
Kernel	its own	shares the host's

ANALOGY

VM = building a separate house for each tenant, each with its own foundation, plumbing and wiring.

Container = apartments in one building, sharing the foundation and plumbing, but with locked doors and fully separate living space.

Containers give you most of the isolation at a tiny fraction of the cost. That trade-off is why the entire cloud industry runs on them.

THE FOLLOW-UP THAT SEPARATES CANDIDATES

"Containers share the host kernel" is the textbook line. Here is the answer of someone who has actually used it:

Containers need a *Linux* kernel. macOS does not have one. So on a Mac, Docker *always* runs a hidden Linux VM, and your containers run inside that. Docker Desktop hides this so thoroughly that many developers don't know. Colima makes it explicit — `colima start` literally starts that VM.

We proved it: running `docker info` on a Mac reported `Operating System: Ubuntu 24.04.4 LTS`.

2.3 The vocabulary

TERM	PLAIN MEANING
Image	A read-only template — the recipe. <code>postgres:17-alpine</code> is an image.
Container	A running instance of an image. One image, many containers.
Volume	Storage that lives <i>outside</i> the container, so data survives the container.
Port mapping	A door from your machine's network into the container's private one.
Registry	Where images are stored and fetched from (Docker Hub by default).
Tag	The version part: <code>postgres:17-alpine</code> . <code>latest</code> is a trap.

Alpine in a tag means the image is built on Alpine Linux, a very small distribution — smaller downloads and a smaller attack surface.

2.4 Docker Compose — declaring a whole stack

Our real `docker-compose.yml`, annotated:

```

services:                                # each "service" becomes one container
  postgres:
    image: postgres:17-alpine            # which image. -alpine = small Linux base
    container_name: reconpilot-postgres  # fixed name, so docker exec is predictable
    environment:                          # config the image reads on FIRST boot
      POSTGRES_DB: reconpilot
      POSTGRES_USER: reconpilot
      POSTGRES_PASSWORD: localdev
    ports:
      - "5432:5432"                      # HOST port : CONTAINER port
    volumes:
      - pgdata:/var/lib/postgresql/data  # persist data outside the container
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U reconpilot -d reconpilot"]
      interval: 5s
      retries: 10

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

volumes:
  pgdata:                                # declares the named volume used above

```

NOTE THERE IS NO **VERSION:** KEY

It was removed in Compose v2 and now produces a warning. Most tutorials online still include it.

Ports

A container has its own private network. `"5432:5432"` maps *your Mac's* port 5432 to the *container's* 5432, which is how Spring Boot running on your Mac reaches Postgres inside the container. The two numbers need not match: `"5433:5432"` would let you run a second Postgres without a clash.

Volumes — containers are disposable, data is not

ANALOGY

A container is a **hotel room**. When you check out it is wiped clean. A volume is the **luggage you take with you** — it exists independently of any room.

```

docker compose down    # stop and remove containers – volume SURVIVES
docker compose down -v # ...and DELETE the volume. Data is genuinely gone.

```


2.7 What Docker gave *this* project

- **Postgres and Redis with no installers** — one file, one command, identical for anyone who clones the repo.
- **Disposable test databases** — Testcontainers starts a real Postgres per test run and destroys it.
- **Version pinning** — `postgres:17-alpine` in both Compose and tests, so the database you test against is the database you deploy.

A REAL BUG THIS CAUSED

Spring Initializr generated the Testcontainers config with `postgres:latest`. Our tests ran against **18.6** while Compose ran **17**. Two problems: you are testing a version you do not deploy, and **the same commit tests differently next month** as `latest` moves. Pinning fixed both.

3 · PostgreSQL

Most of the hard thinking in ReconPilot is in the database. Not because the queries are complex, but because correctness, history and isolation are all enforced here rather than hoped for in application code.

3.1 Money must be integers

This is the single most important rule in financial software, and it is easy to get wrong because the wrong way appears to work.

The demonstration we ran on your machine

```
--- The classic ---
0.1 + 0.2 = 0.30000000000000004
Is it 0.3? false

--- 100,000 transactions of ₹3,333 at 0.4% ---
double : 1333200.0000005632
paise  : 133300000 paise = 1333000.00
exact   : 1333000.00
```

Why $0.1 + 0.2 \neq 0.3$

Computers store numbers in **binary**. Some decimals have no exact binary form — exactly as $\frac{1}{3}$ has no exact decimal form (0.3333... forever). 0.1 in binary repeats forever, so the computer stores the *nearest representable value*. Add two approximations, get an approximation.

This is not a Java bug. It is IEEE 754, and it behaves identically in Python, JavaScript, C# and every other language.

THERE ARE TWO BUGS IN THAT OUTPUT, NOT ONE

Conflating them is a mistake. The trailing `0.0000005632` is **floating-point error** — small, but it means you can never compare money with `==`, and in a reconciliation product comparing amounts is *the entire product*.

The **₹200 gap** is not float error at all. It is a **rounding decision**: 0.4% of ₹3,333 is ₹13.332, and whether that becomes ₹13.33 or keeps the fraction changes the total at scale.

Integers do not eliminate the rounding decision. They force you to make it once, explicitly, and apply it identically every time. Floats make the decision for you, silently, differently depending on the values. That inconsistency is the real crime, not the imprecision.

The rule

```
amount_paise BIGINT NOT NULL CHECK (amount_paise >= 0)
```

All money is `BIGINT` paise. Never `float`, `double`, `REAL` or `DOUBLE PRECISION`. Where division is unavoidable, use `BigDecimal` with an explicit `RoundingMode` defined in exactly one place. `BIGINT` holds about 9.2×10^{18} paise, far beyond any plausible transaction.

Convert at the boundary only — when parsing input and when rendering output. Those two points are where bugs concentrate, so they get concentrated tests.

3.2 Two kinds of time — bi-temporal modelling

Almost nobody gets this right, and it is what makes audit possible.

```
occurred_at  TIMESTAMPTZ NOT NULL,  -- when the payment actually happened
recorded_at  TIMESTAMPTZ NOT NULL    -- when OUR SYSTEM learned about it
```

ANALOGY

A transaction happened on **3 March**. The settlement file carrying it reached you on **10 March**.

If someone asks "*what did you believe on 5 March?*", the honest answer must **exclude** that transaction — on the 5th you genuinely did not know about it. One timestamp cannot express that; two can.

```
-- what is true now
SELECT ... WHERE occurred_at BETWEEN ? AND ?

-- what we BELIEVED at time T (this is the one that matters in a dispute)
SELECT ... WHERE occurred_at BETWEEN ? AND ? AND recorded_at <= T
```

ONE RECORDED_AT PER FILE, NOT PER ROW

Every row from one ingestion shares a single `recorded_at`. We verified this: `SELECT count(DISTINCT recorded_at) over 2,000,000 rows` returned **1**.

Calling `now()` per row would smear one file across time, so a replay filtered on `recorded_at` could include *half* a file — and "what did we know at 10:31" would have no meaningful answer.

3.3 Store facts, derive answers — event sourcing

The instinct is to keep a running total:

```
merchant.monthly_total += transaction.amount;  // every problem lives here
```

One mutable number. No history, no ordering, no explanation. It cannot be replayed, audited or corrected.

ANALOGY — YOUR BANK STATEMENT

Your bank does not store only your balance. It stores every transaction, and the balance is **derived**. That is why you can dispute a charge from three years ago. If banks stored only the current balance, no dispute could ever be settled.

So `transaction_event` is **append-only**: no UPDATE, no DELETE. A restatement from a PSP arrives as a *new row* with the same `external_txn_id` and a later `recorded_at`. The current view is the greatest `recorded_at`; the historical view is the greatest `recorded_at <= T`.

Truth vs cache

TRUTH (append-only, never changes)	CACHE (drop and rebuild anytime)
└─ <code>transaction_event</code>	└─ <code>merchant_month_total</code>
└─ <code>merchant_category_event</code>	└─ <code>mdr_calculation</code>
└─ <code>ingestion_batch</code>	└─ Redis projections

Everything on the right can be deleted and regenerated from the left. That is not merely a nice property — it is a **testable** one: rebuild the projections from scratch, compare to the live ones, and any difference is a bug.

3.4 Indexes — what they cost

ANALOGY

An index is the index at the back of a textbook. Without it, finding "idempotency" means reading every page. With it, you look up one entry and jump straight there.

But the index took paper to print, and **every time the book changes, the index must be reprinted**. That is exactly the trade: faster reads, slower writes, more storage.

We measured it on 2,000,000 real rows:

COMPONENT	SIZE	SHARE
Table data	522 MB	60%
Indexes	349 MB	40%
Total	871 MB	456 bytes/row

MY ESTIMATE WAS 52% LOW

ADR 0002 estimated 300 bytes per row. Reality was **456** — because the estimate counted column data and **ignored indexes entirely**, which turned out to be 40% of storage. That is the classic estimation mistake.

But notice: the estimate was wrong in *detail* and right in *decision*. A single Postgres instance was still correct. **Back-of-envelope arithmetic is for choosing an architecture, not for capacity planning.**

3.5 Keyset pagination, not OFFSET

A classic interview question, and the reason "our admin page gets slow after page 50" is such a common bug report.

```
OFFSET 500000 LIMIT 5000    -- database reads 505,000 rows and throws away 500,000
WHERE id > ? LIMIT 5000    -- index seeks straight to the position
```

ANALOGY

OFFSET is counting pages from the front of the book every single time you want to continue reading. **Keyset** is a bookmark.

OFFSET degrades with *every* page because the database must count past everything it has already returned. Over a million rows that is quadratic work. Keyset remembers where it stopped, so page 200 costs what page 1 cost.

Our reconciliation walks a batch in 5,000-row keyset pages, which is how 1,000,000 rows scan in **3.0 seconds in a 256 MB heap**. The index was widened to `(batch_id, id)` to support it.

Why chunks rather than one long cursor

A single streaming cursor also uses constant memory — but it holds **one database transaction open for the entire run**. Long transactions block vacuum, accumulate write-ahead log, and make the run all-or-nothing with no ability to resume. Chunking gives constant memory *and* short transactions.

3.6 Constraints beat code

ANALOGY

A sign saying "**one person at a time**" is application logic — it works until someone ignores it. A **turnstile** is a database constraint.

If a rule must never be violated, push it as far down the stack as it will go. Application code is where rules go to be forgotten, refactored away, or raced against.

Idempotency by unique constraint

```
UNIQUE (tenant_id, content_hash) -- SHA-256 of the file's bytes
```

Upload the same file twice and **PostgreSQL** rejects it — not an `if` statement someone can forget. Measured: a duplicate 148 MB upload returned in **652 ms** having written nothing.

Partial unique index — when the rule has an exception

```
CREATE UNIQUE INDEX uq_batch_content_not_failed  
ON ingestion_batch (tenant_id, content_hash)  
WHERE status <> 'FAILED';
```

"At most one *non-failed* batch per file, and any number of failed attempts." A plain unique constraint would block retrying a file that failed once, which means one transient error blocks it forever.

FOUND BY THE VERY FIRST INTEGRATION TEST

We changed the application's idempotency *query* to ignore FAILED batches, but left the *constraint* untouched. The code decided to retry; the database refused with a duplicate key.

The same rule was written in two places and only one was changed. A system that contradicts itself.

Upsert — let the database resolve races

```
INSERT INTO merchant (id, tenant_id, external_ref, display_name)  
VALUES (?, ?, ?, ?)  
ON CONFLICT (tenant_id, external_ref)  
DO UPDATE SET display_name = merchant.display_name  
RETURNING id
```

The apparently pointless `DO UPDATE SET x = x` is deliberate: `DO NOTHING` returns no rows, so `RETURNING` would yield nothing on conflict. Assigning the column to itself makes the conflicting row be returned.

This replaced a **check-then-act race** that failed 12 of 14 batches on the first concurrent run — covered in full in §8.

3.7 JSONB — keep the original, always

```
raw JSONB NOT NULL -- the source row, verbatim
```

Every parse is a **lossy interpretation**. When a customer disputes a figure, you argue from what the PSP actually sent, not from your interpretation of it. Storage cost was 178 MB across 2,000,000 rows — being unable to prove your input is unbounded liability.

`JSONB` (binary JSON) is stored parsed and can be indexed; `JSON` stores the raw text. Prefer `JSONB` unless you need to preserve exact formatting and key order.

3.8 Row-level security — the feature that did nothing

RLS lets the database itself filter rows per user, so a forgotten `WHERE` clause cannot leak data.

ANALOGY

Application filtering is a **receptionist checking badges**. RLS is **the floors being locked**. For a multi-tenant financial product you want both, because one leak is company-ending.

OURS WAS ENABLED AND COMPLETELY INERT

```
relname          | rls_enabled | rls_forced
transaction_event | t           | f           ← enabled, NOT forced

current_user: reconpilot      ← also the table owner, and a superuser

SELECT count(*) FROM transaction_event;  -- no tenant set
-> 2000000                                ← everything visible
```

Two independent exemptions, either of which disables RLS completely:

1. PostgreSQL exempts a table's **owner** unless `FORCE` is also set.
2. A **superuser** bypasses RLS entirely, even with `FORCE`.

The app connected as a role that was both. The policies were syntactically perfect and never consulted.

A security control that is enabled but not enforced is worse than none, because it creates false confidence. The schema *reads* as though isolation is handled. A reviewer would believe it.

The fix needs every part

```
-- 1. a least-privilege role: neither owner nor superuser
CREATE ROLE reconpilot_app LOGIN PASSWORD '...';
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO reconpilot_app;

-- 2. force RLS so it applies to the owner too
ALTER TABLE transaction_event FORCE ROW LEVEL SECURITY;

-- 3. policies that govern reads AND writes
CREATE POLICY tenant_isolation ON transaction_event
    USING      (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::uuid)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::uuid);
```

PIECE	WHY IT IS NOT OPTIONAL
FORCE	Without it the owner bypasses every policy.
Non-superuser role	A superuser bypasses RLS even with FORCE. This is a precondition, not hardening.
WITH CHECK	USING governs reads only. Without this a tenant could write rows carrying another tenant's id — writable but invisible, which is worse than readable.
, true	"missing_ok": returns NULL when unset rather than raising an error.
NULLIF(..., '')	Cleared settings become '', and '::uuid raises. NULLIF collapses both cases to NULL, and a NULL comparison is never true — so "no tenant" means "no rows", which is the safe default.

The proof

```
reconpilot_app, no tenant set → 0 rows
reconpilot_app, tenant = Acme → 300 rows
reconpilot_app, tenant = Globex → 120 rows
knowing another tenant's row id → 0 rows
```

3.9 Connection pooling, and the leak it creates

Opening a database connection is slow — TCP handshake, authentication, session setup. So Spring opens a handful up front and reuses them. That is **HikariCP**, and "why pool connections?" is a common interview question: *because the handshake costs more than the query.*

POOLING AND SESSION VARIABLES ARE A DANGEROUS COMBINATION

A PostgreSQL session variable set on a pooled connection **stays on it** when it returns to the pool. The next request to borrow that connection inherits the previous request's tenant.

That is a cross-tenant data leak caused by an optimisation that has nothing to do with tenancy — and it would be completely silent.

Our defence is doubled on purpose: set the tenant on *every checkout* (so whatever the last borrower left is overwritten) *and* clear it on close (so nothing goes back carrying state). Either alone would suffice in the normal path; both are there because the failure is silent and severe.

3.10 Flyway — migrations as version control for the schema

```
V1__initial_schema.sql
V2__batch_lifecycle_columns.sql
V3__recon_break_idempotency.sql
V4__allow_retry_of_failed_batches.sql
V5__dispute_workflow.sql
V6__auth_and_enforced_tenant_isolation.sql
```

The filename is a convention: `V<version>__<description>.sql` with **two** underscores. Flyway applies them in order and records each in `flyway_schema_history`, so it never runs one twice.

TWO SETTINGS THAT MATTER

```
spring.flyway.enabled=true
spring.jpa.hibernate.ddl-auto=validate
```

Flyway owns the schema. Hibernate only checks it. `validate` means Hibernate compares your entities to the schema and complains if they disagree, but never alters anything.

The tempting alternative, `ddl-auto=update`, lets Hibernate silently change your *production* schema based on your Java classes. Never do that. "*Flyway owns schema, Hibernate validates only*" is the answer an interviewer wants.

Migrations run as the **owner**; the application runs as the least-privilege role. Splitting them is the point — with one all-powerful connection, RLS cannot work at all.

4 • Spring Boot

Spring is large, but it rests on a small number of ideas. Understand those five and the rest is vocabulary.

4.1 Inversion of Control — the one idea everything else sits on

Normally *your* code decides what it needs and creates it:

```
class OrderService {  
    private final EmailSender email = new SmtplibEmailSender("smtp.gmail.com", 587);  
    //                                     ↑ this class now DEPENDS on SMTP forever  
}
```

Want to test it without sending real email? You can't. Want to swap to SendGrid? Edit the class.

ANALOGY — HIRING

Without IoC: every employee recruits their own assistant, negotiates their salary and buys their laptop. Chaos, and nobody can be replaced.

With IoC: HR hires people and hands each employee whoever they need. The employee says "*I need an assistant*" and one arrives. Swapping assistants is HR's problem, not theirs.

Spring is HR. "Inversion of control" means the control over *who creates what* is inverted — moved out of your class and into the framework.

```
class OrderService {  
    private final EmailSender email;  
    OrderService(EmailSender email) { this.email = email; }    // just ask  
}
```

Now the class works with any `EmailSender`. In a test you pass a fake. In production Spring passes the real one. This is **dependency injection**, and it is why Spring code is testable.

4.2 Beans — the objects Spring manages

A **bean** is simply an object Spring created and holds on to. Four ways to declare one:

ANNOTATION	MEANING	WE USED IT FOR
<code>@Component</code>	generic managed object	<code>SettlementCsvParser</code>
<code>@Service</code>	business logic (a <code>@Component</code> with intent)	<code>IngestionService</code> , <code>DisputeService</code>
<code>@RestController</code>	handles HTTP, returns data	<code>MdrController</code>
<code>@Bean</code> in <code>@Configuration</code>	you build it yourself	<code>MdrCalculator</code> , the <code>DataSources</code>

WHY MDRCALCULATOR IS A @BEAN AND NOT A @SERVICE

This was deliberate and it is a strong interview talking point.

```
@Configuration
public class DomainConfig {
    @Bean MdrCalculator mdrCalculator() { return new MdrCalculator(); }
}
```

Putting `@Service` on `MdrCalculator` would place a Spring annotation **inside the domain logic**. Declaring it as a `@Bean` elsewhere keeps the class plain Java.

The payoff is measurable: **23 calculator tests run in under 20 ms** with no Spring context, while the one test that starts Spring takes 2.5 seconds.

The framework should know about your domain. Your domain should not know about the framework.

That is **dependency inversion**, the core idea behind hexagonal and clean architecture.

Constructor injection, always

```
public MdrController(MdrCalculator calculator) { this.calculator = calculator; }
```

One constructor means Spring needs no annotation to know what to inject. Fields can be `final`, so the object cannot exist in a half-built state, and the dependencies are visible in the signature. Field injection (`@Autowired` on a field) hides them and makes the class untestable without Spring.

4.3 Auto-configuration — and how adding a bean can *remove* one

Spring Boot looks at your classpath and configures things automatically. Jackson present? JSON conversion is wired. A JDBC driver and a URL? A `DataSource` and a `JdbcTemplate` appear.

It does this with **conditional** configuration — most importantly:

```
@ConditionalOnMissingBean(JdbcOperations.class) // "only if the user hasn't made one"
```

THIS COST US HOURS, AND IT IS A GENUINELY GREAT INTERVIEW STORY

We declared an admin `JdbcTemplate` for cross-tenant system work. Spring Boot saw a `JdbcOperations` bean already existed and **silently stopped creating the normal one**.

Result: every controller and service got the **privileged** template. Row-level security worked perfectly at the database level and applied to nothing, because no application query ever reached it.

Adding a bean removed another one. No warning. The app started fine. The tests passed. Only an end-to-end check with two real tenants exposed it.

The fix: declare the normal one explicitly and mark it `@Primary`.

4.4 Proxies — the trap in `@Async`, `@Transactional` and `@Cacheable`

How does `@Transactional` actually start a transaction? Spring **wraps your bean in a proxy** — a stand-in object that looks identical from outside but does extra work before and after each call.

ANALOGY — THE RECEPTIONIST

A receptionist sits outside your office. Visitors from **outside** must pass them, and the receptionist says *"he's busy — I'll put this in his inbox for later"* (that is the async behaviour).

But if you are **already inside** the office and walk to the next desk, you never pass the receptionist. You just interrupt that person directly — **synchronously**.

```
class Service {
    void submit() {
        this.process();    // ← runs RIGHT NOW. @Async is silently ignored.
    }
    @Async
    void process() { /* long work */ }
}
```

NOTHING WARNS YOU

It compiles, the annotation is right there, and it simply is not async. That is why `IngestionWorker` is a **separate bean** — the call comes from outside, so it passes the proxy.

The same trap applies to `@Transactional` and `@Cacheable`. This is one of the most common Spring interview questions.

4.5 Building a REST API

```
@RestController
public class MdrController {

    @GetMapping("/api/mdr")
    public MdrResult calculate(
        @RequestParam long amountPaise,
        @RequestParam TxnType txnType,          // Spring converts String → enum
        @RequestParam PaymentRail rail,
        @RequestParam PayeeCategory payeeCategory) {
        return calculator.calculate(new MdrInput(amountPaise, txnType, rail, payeeCategory));
    }
}
```

ANNOTATION	READS FROM	EXAMPLE
@RequestParam	query string	?amountPaise=300000
@PathVariable	the URL path	/api/disputes/{id}
@RequestBody	the request body (JSON)	the login form
@GetMapping	GET — "give me something"	safe, repeatable
@PostMapping	POST — "do something"	changes state

WHY /API/INGEST CANNOT BE TESTED IN THE BROWSER ADDRESS BAR

Typing a URL always sends a **GET**. Ingestion is a **POST** because it changes state, so the browser gets 405 Method Not Allowed. That is why /api/mdr works in a browser and /api/ingest needs curl.

How a Java object becomes JSON

```
@RestController = @Controller + @ResponseBody
|
"do not treat my return value as a page name –
serialise it into the response body"
|
Spring picks an HttpMessageConverter → Jackson
|
reads the record's accessors mdrPaise() and ruleId()
|
{"mdrPaise":1200,"ruleId":"STANDARD_0P4_CAPPED"}
```

You write no conversion code because Spring Boot auto-configured it the moment Jackson appeared on the classpath. **That is "convention over configuration" again** — the same idea from Maven, now in Spring.

Records — Java's data holders

```
public record MdrResult(long mdrPaise, String ruleId) { }
```

Replaces about 50 lines: fields, constructor, getters, `equals`, `hashCode`, `toString`. Immutable by default, which is exactly right for money.

4.6 Error handling — status codes carry meaning

```
@RestControllerAdvice          // applies to EVERY controller
public class ApiExceptionHandler {

    @ExceptionHandler(UnspecifiedRuleException.class)
    @ResponseStatus(HttpStatus.NOT_IMPLEMENTED)    // 501
    public Map<String,String> handleUnspecifiedRule(UnspecifiedRuleException e) {
        return Map.of("error", "RULE_NOT_SPECIFIED", "message", e.getMessage());
    }
}
```

STATUS	MEANS	WHERE WE USE IT
200	here is your result	a computed MDR
201	created	register a workspace
202	accepted, not done	file upload queued
400	your request was malformed	negative amount, bad enum
401	we don't know who you are	no or invalid token
403	we know you, you may not	another tenant's row
409	conflicts with current state	illegal dispute transition
422	understood, forbidden by a rule	claiming an undercharge
501	we genuinely cannot serve this	education rate not published
503	at capacity, retry	ingestion pool full

WHY 501 AND NOT 500 FOR AN UNKNOWN RULE

500 means "we are broken." But we are not — we deliberately refuse to invent a rate the regulator never published. Correct behaviour reported as a crash.

A customer seeing 500 files a bug report. A customer seeing `501 {"error":"RULE_NOT_SPECIFIED"}` understands the gap is in the *regulation*, not your software. Same outcome, completely different trust.

401 VS 403 — A DISTINCTION MOST APIS GET WRONG

Spring Security's *default* answers **403** to a caller with no credentials at all. That is the wrong distinction: a client cannot tell "log in again" from "you lack permission" if both are 403. We configured an entry point returning 401.

4.7 Data access — JdbcTemplate and JPA

	JPA / HIBERNATE	JDBCTEMPLATE
Built for	a graph of mutable domain objects with a lifecycle	plain SQL
Per-row overhead	high (persistence context grows)	minimal
Good for	CRUD on rich entities	bulk writes, reporting queries

WE CHOSE JDBCTEMPLATE FOR THE EVENT LOG DELIBERATELY

These rows are **immutable facts written once and never updated**. An ORM would add per-row overhead *and* a persistence context that grows with the file — reintroducing the exact memory problem streaming had just solved.

Use an ORM for domain objects with a lifecycle. Use plain SQL for bulk immutable writes.

Knowing when *not* to reach for the framework is the senior signal.

Batching, and a free 28%

```
jdbcTemplate.batchUpdate(INSERT_EVENT, buffer); // 1,000 rows per round trip
```

Then one connection-string parameter, no code change:

```
jdbc:postgresql://localhost:5432/reconpilot?rewriteBatchedInserts=true
```

2,000,000 ROWS	TIME	ROWS/SEC
JDBC batching	70.8 s	~28,000
+ <code>rewriteBatchedInserts</code>	51.0 s	~39,000

The driver collapses a batch into one multi-row `INSERT` instead of N single-row ones. **Measured, not assumed** — that is the habit worth keeping.

4.8 Configuration

```
reconpilot.jwt.secret=${RECONPILOT_JWT_SECRET:local-development-secret-...}
reconpilot.claims.filing-enabled=false
```

`${VAR:default}` reads an environment variable and falls back. That is how a secret stays out of the repository while local development still works.

```
@Value("${reconpilot.claims.filing-enabled:false}") boolean filingEnabled // one value
@ConfigurationProperties("app.datasource.admin") // a whole group
```

4.9 Asynchronous work and bounded capacity

```
@Bean("ingestionExecutor")
public ThreadPoolTaskExecutor ingestionExecutor() {
    var ex = new ThreadPoolTaskExecutor();
    ex.setCorePoolSize(2);
    ex.setMaxPoolSize(4);
    ex.setQueueCapacity(10);
    ex.setRejectedExecutionHandler(new ThreadPoolExecutor.AbortPolicy());
    return ex;
}
```

ANALOGY — THE RESTAURANT

2 cooks normally, up to 4 when busy, 10 seats in the waiting area.

No limit on cooks → hire one per order → 10,000 cooks → the restaurant collapses.

No limit on seats → accept every booking forever → the building falls down. This one is nastier, because it *looks* fine for hours, then dies of an `OutOfMemoryError` far from the actual cause.

A GENUINE SURPRISE IN THREADPOOLTASKEXECUTOR

It fills the **queue before** hiring extra threads:

```
2 threads busy → next tasks go on the queue (up to 10)
queue full     → NOW create threads 3 and 4
```

Most people assume it grows the pool first. It does not. So with a 10-slot queue, thread 4 does not appear until 12 tasks are outstanding.

ABORTPOLICY, NOT CALLERRUNSPOLICY

CallerRuns hands the work back to the *HTTP thread* — silently restoring the synchronous behaviour we removed, at the exact moment the system is busiest. The counter stops being served entirely.

Abort surfaces overload honestly as **503** and lets the client retry. Measured: 16 concurrent uploads against a capacity of 14 produced exactly **14 accepted, 2 rejected**. That is **backpressure** — a system that knows its own limit and says so.

4.10 Spring Security and JWT

What a JWT actually is

Three base64 segments: a header, a payload of claims, and a signature over the first two.

THE PAYLOAD IS ENCODED, NOT ENCRYPTED

Anyone holding the token can read it. What the signature guarantees is that nobody has **changed** it without the secret. So a token may carry an id and a role, and **must never carry a secret**.

ANALOGY — SESSIONS VS TOKENS

Session id = a cloakroom ticket. The number means nothing by itself; the cloakroom must remember which coat it maps to. Every attendant needs the same list.

JWT = a signed, tamper-evident boarding pass. Everything needed is printed on it, and the signature proves it is genuine. Any gate can check it with no lookup and no shared store.

The cost: a boarding pass cannot be un-issued. Until it expires it stays valid, even if the passenger is banned — which is why expiry is short and instant revocation needs a denylist on top.

Password hashing

```
@Bean PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
```

BCrypt, never SHA-256 or MD5. Fast hashes are the wrong tool for passwords *precisely because they are fast* — a GPU tries billions of guesses per second. BCrypt is deliberately slow, its cost is tunable upward as hardware improves, and it **salts** each password so two users with the same password get different hashes, which defeats precomputed rainbow tables.

The filter chain

```
.csrf(csrf -> csrf.disable()) // no cookies = no CSRF vector
.sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/auth/**").permitAll()
    .requestMatchers("/actuator/health").permitAll()
    .anyRequest().authenticated() // DEFAULT DENY
.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
```

DEFAULT DENY

`anyRequest().authenticated()` means a new endpoint is **protected because nobody remembered to open it**, rather than exposed because nobody remembered to close it. Always order your rules that way.

Two security details worth stating in an interview

- **The tenant comes from the signed token, never from a parameter.** A `?tenantId=` parameter would let anyone read anyone's data by editing a URL — among the most common multi-tenant vulnerabilities there is.
- **Unknown email and wrong password return the same 401.** Distinguishing them turns the login form into a way to discover which email addresses have accounts.

ThreadLocals and why they must be cleared

```
try {
    // ... establish tenant from token, handle request
} finally {
    TenantContext.clear();
    SecurityContextHolder.clearContext();
}
```

Request threads are **pooled and reused**. A tenant left in a ThreadLocal becomes the *next* request's tenant — a cross-tenant leak arising from thread reuse rather than from any mistake in business logic.

A THREADLOCAL DOES NOT CROSS A THREAD BOUNDARY

The ingestion worker runs on a *pool* thread, where the request thread's ThreadLocal is simply absent. That is why the tenant id is passed to it as an explicit parameter and re-established at the start of the task.

4.11 Actuator

```
management.endpoints.web.exposure.include=health,info
```

`/actuator/health` returns `{"status":"UP"}` and is what a load balancer or Kubernetes polls to decide whether to send you traffic. Expose deliberately: the full set includes endpoints that dump configuration and environment variables.

4.12 Reading a Spring Boot startup log

Starting BackendApplication using Java 21.0.12.1	← which JDK
Bootstrapping Spring Data JPA repositories	← scanning for @Repository
Found 0 JPA repository interfaces	← we have none. Correct.
Tomcat initialized with port 8080	← the web server
HikariPool-1 – Added connection	← DB connection pool opened
Successfully validated 1 migration	← Flyway checked
Database version: 17.11	← Postgres in the container
Started BackendApplication in 1.317 seconds	← ready

5 • Testing

87 tests. But the number is not the point — three of them were verified to *fail* against the original bugs, and that is what makes them worth having.

5.1 Unit vs integration — and why the split is real

	UNIT TEST	INTEGRATION TEST
Touches	one class, no framework	real database, real Spring context
Speed	0.001–0.02 s	0.5–3 s
Needs Docker	no	yes
Our naming	<code>*Test</code>	<code>*IT</code>
Run by	Surefire (<code>mvn test</code>)	Failsafe (<code>mvn verify</code>)

I WROTE TESTS THAT COULD NEVER RUN

The build was green at 35 tests. `IngestionIT` and `ReconciliationIT` compiled and were silently skipped.

Maven Surefire only runs classes matching `*Test`, `Test*` or `*Tests`. Anything ending `IT` is conventionally an *integration* test, handled by the **Failsafe** plugin — which was not configured.

A green build told me nothing, because the tests I cared about never executed. Nothing warns you.

```
./mvnw test      → 52 unit tests, seconds, no Docker    (fast feedback)
./mvnw verify    → + 35 integration tests              (the full check)
```

A "UNIT TEST" THAT NEEDED DOCKER

Spring Initializr generates `BackendApplicationTests`, which starts the entire application — and therefore a database container. But the `*Tests` name put it in the fast phase, so `mvn test` required Docker.

A test that needs a database is not a unit test.

Renamed to `ApplicationContextIT` so Failsafe runs it. Now `mvn test` genuinely needs nothing.

5.2 Testcontainers — a real database, thrown away

```
@Bean @ServiceConnection
PostgreSQLContainer postgresContainer() {
    return new PostgreSQLContainer(DockerImageName.parse("postgres:17-alpine"));
}
```

Starts a genuine PostgreSQL in Docker on a random port, runs your migrations against it, and destroys it afterwards. Your tests never depend on the state of your dev database, and never corrupt it.

Ryuk is a small janitor container Testcontainers runs to clean up if your test crashes. On Colima it needs the socket path *as seen from inside the VM*, which is why

`TESTCONTAINERS_DOCKER_SOCKET_OVERRIDE=/var/run/docker.sock` exists — a concrete example of the container/host boundary being real.

5.3 The test that destroyed a database

THIS IS THE MOST IMPORTANT LESSON IN THE CHAPTER

`@ServiceConnection` only overrides `spring.datasource.*`. Our hand-rolled `app.datasource.admin.*` kept pointing at `localhost:5432/reconpilot` — **the real development database** — and `clearDatabase()` ran `TRUNCATE ... CASCADE` against 1,000,000 rows of real data.

A test suite that can reach your development database will eventually destroy it. Not might — will.

There was a second cause underneath: **replacing Spring Boot's auto-configured DataSource with a hand-built one silently disconnects `@ServiceConnection` entirely**, because Testcontainers contributes a `JdbcConnectionDetails` bean that only the *auto-configured* DataSource consumes.

The fix, in two layers

```
// 1. point every datasource at the container
registry.add("app.datasource.admin.url", postgres::getJdbcUrl);
registry.add("spring.flyway.url",          postgres::getJdbcUrl);

// 2. refuse to run against anything else
if (!url.equals(postgres.getJdbcUrl())) {
    throw new IllegalStateException("Refusing to run: not the test container");
}
```

A destructive operation should verify what it is about to destroy, rather than trust that the wiring is right.

Verified with a canary row in the dev database that survived a full suite run.

5.4 Test-first, and why it actually matters

We wrote the MDR tests from the regulator's published figures **before** the calculator existed. One failed:

```
capBindsExactlyAtSeventyFiveThousand
```

```
"below Rs 75,000 the percentage governs" ==> expected: <true> but was: <false>
```

₹74,999 × 0.4% = ₹299.996, which **HALF_UP** rounds to **₹300** — exactly the cap. My assumption was wrong; the code was right.

WHY WRITING THE TEST FIRST MATTERED HERE

Had I written the code first and the test after, I would have run the calculator, seen ₹300, thought "right, that's what it does", and asserted ₹300. **The test would have passed and taught me nothing.**

Tests written after the code tend to encode the code's behaviour rather than the requirement — so they lock in bugs instead of catching them.

It also found something real: the percentage and the cap **interact through the rounding mode**. Nobody could predict that by reading the regulation.

5.5 Validating a detection engine — plant known errors

You cannot validate a detection engine against data whose correct answer you do not know. Every output looks equally plausible. So the test file is generated with a **known number of deliberately corrupted rows**.

BREAK TYPE	PLANTED	FOUND
CHARGED_WHEN_EXEMPT	1,500	1,500
CAP_BREACHED	800	800
OVERCHARGED	3,000	3,000
UNDERCHARGED	2,000	2,000
Total	7,300	7,300

THE GENERATOR WAS AN INDEPENDENT IMPLEMENTATION, AND THAT WAS THE POINT

The expected answers were computed in Python, by hand — not by calling the Java code.

The first run found **15,474** breaks instead of 7,300. All 8,174 extras were capital-markets rows off by **exactly one paise**: the generator truncated (66.66 → 66) where the calculator rounds HALF_UP (66.66 → 67).

A test that asks the code whether the code is right is a tautology.

Had the expectations been generated from `MdrCalculator`, the test would have passed perfectly and proved nothing.

5.6 A regression test must be seen to fail

We put both original bugs **back** and ran the tests:

Bug: check-then-act merchant creation restored

→ concurrentBatchesSharingMerchantsDoNotCollide FAILED: DuplicateKeyException ✓

Bug: rejection leaves the batch row restored

→ rejectionLeavesNoTraceSoTheFileCanStillBeIngested

FAILED: "a refused request must leave no trace" expected: <0> but was: <1> ✓

Then restored both fixes — `git diff` confirmed byte-identical.

A regression test that has never failed is a hypothesis, not a test. You have asserted it catches the bug. Until you watch it catch the bug, you do not know.

5.7 JUnit essentials

<code>@Test</code>	a test method
<code>@BeforeEach / @AfterEach</code>	run before/after EVERY test
<code>@Nested</code>	group related tests, with their own <code>@DisplayName</code>
<code>@DisplayName("...")</code>	a readable name in the report
<code>@ParameterizedTest</code>	run the same test with many inputs
<code>@CsvSource({"3000", "12"})</code>	supply those inputs inline

`assertEquals(expected, actual, "message shown on failure")`

`assertThrows(SomeException.class, () -> code())`

`assertDoesNotThrow(() -> code())`

A COMPLETENESS TEST WORTH COPYING

```
@ParameterizedTest
@EnumSource(DisputeStatus.class)
void everyStateDeclaresItsTransitions(DisputeStatus s) {
    assertNotNull(s.allowedNext(), s + " has no transition entry");
}
```

Add a state to the enum without declaring its transitions and this fails immediately — rather than throwing a `NullPointerException` on whichever unlucky production request meets it first.

6 · React and TypeScript

Three screens, one design system, and a state layer that is Redux without the misery.

6.1 Vite — the build tool

Vite serves your source directly to the browser during development using native ES modules, so a change appears almost instantly (**hot module replacement**) rather than rebuilding a bundle. For production it bundles properly.

```
npm run dev      development server on :5173, hot reload
npm run build    type-check + production bundle into dist/
npm run preview  serve the production build locally
```

6.2 CORS, and why we avoided it entirely

WHAT CORS ACTUALLY IS

Your React app runs at `localhost:5173`. Your API is at `localhost:8080`. To a browser those are **different origins**, and it will refuse the request unless the server explicitly opts in with CORS headers.

This is a browser safety rule, not a server one — `curl` is unaffected, which is why an API can work perfectly in a terminal and fail in a browser.

```
server: {
  proxy: { '/api': { target: 'http://localhost:8080', changeOrigin: true } }
}
```

Now the browser sees **one origin**, so the problem does not arise. And it mirrors production, where a single reverse proxy serves both. **Sidestepping a problem is often better than configuring around it.**

6.3 Components, props and hooks

A component is a function that returns markup. Props are its inputs.

```
function StatCard({ label, value, loading }: Props) {
  return <Paper>{loading ? <Skeleton/> : value}</Paper>
}
```

HOOK	WHAT IT DOES
<code>useState</code>	local state that triggers a re-render when it changes
<code>useEffect</code>	run a side effect after render
<code>useSelector</code>	read a value from the Redux store
<code>useDispatch</code>	send an action to the store
<code>useParams</code>	read <code>:id</code> from the URL
<code>useNavigate</code>	navigate in code

6.4 TypeScript — errors at compile time, not runtime

```
export interface BreakView {
  id: string
  breakType: BreakType
  deltaPaise: number
  disputeId: string | null // explicitly nullable
}
```

Mistype `b.deltPaise` and the build fails, rather than rendering `undefined` to a user. Our `types.ts` mirrors the backend records by hand, so a backend change surfaces as a compile error rather than as a blank screen.

6.5 Redux Toolkit and RTK Query

WHY PLAIN REDUX HAS A BAD REPUTATION

Hand-written, one API call needs an action, a reducer and three state flags (loading, error, data). Times every endpoint. That boilerplate is most of why people dislike Redux.

RTK Query is Redux's own data-fetching layer. You declare endpoints; it generates typed hooks, a normalised cache, loading and error state, and refetching.

```
createDispute: build.mutation({
  query: ({ breakId }) => ({ url: '/disputes', method: 'POST', params: { breakId } }),
  invalidatesTags: ['Break', 'Dispute'], // ← the clever part
})
```

Raise a claim and both the break queue and the dashboard refresh — neither of them knowing a claim was raised. That is tag-based cache invalidation.

SERVER STATE AND CLIENT STATE ARE DIFFERENT THINGS

RTK Query owns what the backend says. A small `uiSlice` owns what *this browser* is currently doing — the active filter, the toast. Mixing them is how caches go stale and filters vanish on refetch.

The token, attached in one place

```
prepareHeaders: (headers, { getState }) => {
  const token = (getState() as RootState).auth.token
  if (token) headers.set('Authorization', `Bearer ${token}`)
  return headers
}
```

Doing this per call is how one endpoint ends up unauthenticated and nobody notices. And a base-query wrapper signs the user out on *any* 401 — since a JWT cannot be un-issued, an ordinary call failing is the first sign it is no longer good.

6.6 Material UI — one design system, not two

WHY WE DID NOT ALSO USE BOOTSTRAP

They are **two complete, competing design systems**. Each ships its own CSS reset, grid, typography scale, spacing scale and components. Using both means conflicting resets, a Bootstrap button sitting next to a Material button looking like two different apps, ~250 KB you do not use, and a constant low-grade decision: "*is this a Bootstrap card or a Material card?*"

That is the *crowded* outcome, not the advanced one. It also reads to an interviewer as "*did not decide*".

With one system, consistency becomes a property of **the configuration** rather than of each developer's discipline. Every colour, radius and font lives in `theme.ts`.

MUI v9 gotcha

System props were removed from `Stack` and `Typography`; layout now goes through `sx`. The compiler errors are long and not obvious:

```
<Stack direction="row" flexWrap="wrap">                                x v9
<Stack direction="row" sx={{ flexWrap: 'wrap' }}>                      ✓
```

6.7 The client mirrors the state machine but is not the authority

```
{ALLOWED_NEXT[claim.status].map(target => <Button>{target}</Button>)}
```

A UI that hides a button has made an action inconvenient, not impossible. Anyone with curl can still try it.

Client-side validation is for **convenience**; server-side is for **correctness**. Getting that backwards is a genuine security bug. The backend still answers 409 for anything illegal.

7 • System design

These ideas outlive the frameworks. They are what an interviewer is actually probing for.

7.1 The six-step framework

Every system design interview is the same six steps. Most candidates fail by jumping straight to step 5.

- | | |
|---------------------|---|
| 1. Requirements | What must it do? What must it GUARANTEE? |
| 2. Scale estimation | How big is this, actually? (do the arithmetic out loud) |
| 3. Data model | What are the nouns, and how do they relate? |
| 4. API design | How does the outside world talk to it? |
| 5. Architecture | What are the boxes and arrows? |
| 6. Deep dive | Attack the hardest part |

Functional vs non-functional

Juniors list features. Seniors list **guarantees**.

REQUIREMENT	WHAT IT MEANT HERE
Correctness	Paisa-exact. A wrong number destroys customer trust instantly.
Auditability	Every figure reproducible and explainable months later.
Idempotency	Uploading the same file twice must not double-count.
Determinism	Same inputs → same outputs, in any arrival order.
Tenant isolation	Merchant A must never see Merchant B's data.
Availability	<i>Low priority.</i> Two hours down and nobody dies.
Latency	<i>Low priority.</i> This is batch work, not a payment gateway.

RANKING TWO REQUIREMENTS LOW IS THE MOST IMPORTANT MOVE

A candidate who says "*it must be highly available, low latency, strongly consistent and infinitely scalable*" has said nothing.

You cannot optimise for everything. Design is choosing what to sacrifice.

Saying "*I traded availability for correctness, because this is financial reconciliation and a wrong number costs more than an outage*" demonstrates judgement — which is the thing being tested.

7.2 Back-of-envelope estimation

$1,000 \text{ merchants} \times 1,000 \text{ transactions/month} = 1,000,000/\text{month} = 12,000,000/\text{year}$

average write rate = $1,000,000 \div (30 \times 86,400) \approx 0.4 \text{ writes/second}$

peak (1M in 1 hour) = $1,000,000 \div 3,600 \approx 280 \text{ writes/second}$

storage $\approx 5.5 \text{ GB/year}$ (measured, including indexes)

READ THOSE NUMBERS AND STOP

0.4 writes per second. A single PostgreSQL instance handles this without effort, for years. No sharding. No microservices. No Kubernetes. No read replicas.

This is not a scale problem. It is a correctness problem.

Ask a junior to design this and you get microservices, Kafka everywhere and a service mesh — architecture cargo-culted from blog posts about Netflix. Netflix has that architecture because Netflix streams to 300 million people. You have 0.4 writes per second.

Complexity is a budget. Spend it where the difficulty actually is.

7.3 Memory must be independent of input size

Measured on a 148 MB, 2,000,000-row CSV with the heap capped:

HEAP	READ WHOLE FILE	STREAM IT
256 MB	251 MB used — survived by 5 MB	65 MB
128 MB	OutOfMemoryError	80 MB
96 MB	OutOfMemoryError	24 MB
32 MB	—	23 MB, still worked

Reading it whole costs $\sim 2.7\times$ the file size, because each line becomes a `String` object carrying about 40 bytes of overhead on top of its characters. **A 148 MB file becomes ~ 400 MB of Java objects.**

THE 256 MB ROW IS THE DANGEROUS ONE

It **worked**. 251 of 256. It would have passed tests, passed review, shipped, and run fine for months. Then the file grows, and one night it dies — at 3 a.m., on your largest customer's largest file, which is exactly when the data matters most.

A bridge that has held every truck you have driven over it is not rated for anything. It is untested.

The principle: memory use must be $O(1)$, not $O(n)$. "*It works on my test file*" is not a capacity statement.

7.4 Idempotency

ANALOGY

Pressing a lift button five times does not summon five lifts. The **first** press changes the state; the rest are no-ops. That is idempotency.

It matters because networks retry. A client sends a request, the response is lost, the client retries — and without idempotency you have ingested the file twice.

Ours is a SHA-256 of the file bytes under a unique constraint. Enforced by the database, not by an `if`. Three separate places in this project use the same pattern: file ingestion, break creation, and dispute creation.

7.5 Precedence is a contract, not a detail

This appeared **three times** in ReconPilot, which is why it is worth naming.

1. P2P? → ₹0
2. Credit-linked? → out of scope
3. AutoPay? → ₹0
4. Merchant is P2PM? → ₹0 ← MUST come before the amount check
5. Amount ≤ ₹2,000? → ₹0
6. Otherwise by category

SPECIFICITY IS NOT PRECEDENCE

Given a ₹2,000 AutoPay payment to a fuel merchant who is P2PM, **four** rules fire. Three say ₹0 and one says ₹5. The instinct is "*the most specific rule wins*" — that is how CSS works. Here it is wrong.

Three rules produce ₹0, so the order looks academic. It is not: **we have to explain the number**. "₹0 because it is AutoPay" and "₹0 because the merchant is P2PM" have different lifespans — if the merchant is reclassified, the AutoPay reason survives and the P2PM reason evaporates.

Which rule fired is as important as what it produced.

7.6 State machines

```
DRAFT --file→ FILED --ack→ ACKNOWLEDGED --accept→ ACCEPTED --settle→ RECOVERED
      |           |           |
      | withdraw | reject  → REJECTED / WITHDRAWN (terminal)
```

The legal transitions live in the enum as **data**, not as `if` statements spread through a service:

```

ALLOWED.put(DRAFT, EnumSet.of(FILED, WITHDRAWN));
ALLOWED.put(ACCEPTED, EnumSet.of(RECOVERED));
ALLOWED.put(RECOVERED, EnumSet.noneOf(DisputeStatus.class)); // terminal

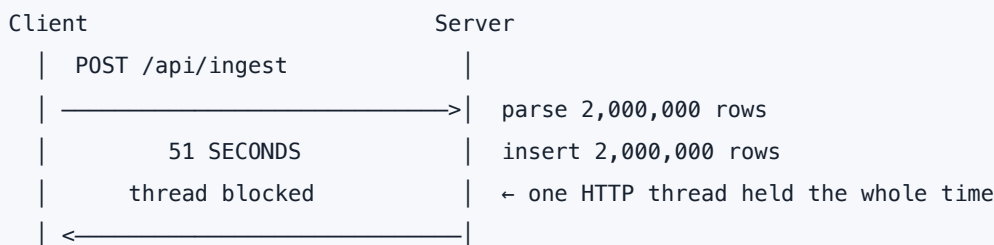
```

Three consequences: the whole lifecycle is readable in one place; adding a state *forces* you to declare its transitions; and an illegal transition becomes **impossible by construction** rather than merely unlikely.

The alternative — a mutable `status` string updated wherever convenient — is how records end up RECOVERED without ever having been FILED.

One gate. There is deliberately no `/file`, `/accept`, `/reject` endpoint: separate endpoints tempt each to do its own checking, and the fifth one, added in a hurry, forgets a rule the other four enforce.

7.7 Async, 202, and backpressure



Four things are wrong, and none is about speed:

1. Proxies and load balancers time out at 30–60 seconds — the request dies *after* most of the work.
2. The client learns nothing until the end. No id, no progress, nothing to poll.
3. A dropped connection strands the batch in `PARSING` forever.
4. **One upload occupies a request thread for a minute.** Tomcat's pool is bounded (200 by default), so a handful of concurrent uploads stops the API answering *anything* — including health checks.

Point 4 turns a slow feature into an outage. The fix took the response from **51,018 ms to 657 ms**.

202, NOT 200

200 claims a result the caller does not have. 202 Accepted says "valid, scheduled, not done", with a `Location` header pointing at where the answer will appear. Returning 200 for work that has not happened is a lie.

7.8 Caching is not always Redis

```

Map<String, UUID> merchantCache = new HashMap<>();

```

2,000,000 rows, 500 distinct merchants. A plain `HashMap` between the parser and the database eliminated **99.975%** of the lookups. That is caching. No infrastructure required.

The general shape: **identify repeated identical work, do it once, remember the answer**. Redis matters when the cache must be shared between processes or survive a restart — not before.

7.9 Concurrency bugs are invisible until you run things concurrently

```
Thread A: SELECT merchant 'shop000@upi' → not found
Thread B: SELECT merchant 'shop000@upi' → not found ← both looked first
Thread A: INSERT → OK
Thread B: INSERT → UNIQUE violation → whole batch dies
```

Both threads checked honestly. Both got a true answer. The answer simply stopped being true in the gap between checking and acting.

2,000,000 rows sequentially: fine. The first concurrent burst: 12 of 14 batches failed.

Any "check, then act" has a gap. If two things can run at once, something will eventually land in it. Make it one indivisible step instead.

7.10 Defence in depth

Two independent mechanisms for anything whose failure is catastrophic:

RISK	LAYER 1	LAYER 2
Cross-tenant data leak	application filters by tenant	PostgreSQL row-level security
Session variable leaking via pool	set on every checkout	clear on close
Duplicate ingestion	application checks the hash	unique constraint
Illegal dispute transition	UI offers only legal buttons	server returns 409

In each pair, layer 1 is convenience and layer 2 is correctness. Where the failure is silent and severe, the cost of the second layer is trivial compared with the cost of being wrong.

8 · Every bug, and what it taught

Ten real defects. For an interview, these are worth more than the features.

8.1 The rounding disaster — 8,174 false claims from one paise

D7 — THE MOST EXPENSIVE KIND OF BUG

The first reconciliation found **15,474** breaks against 7,300 planted. Every extra one was a capital-markets row off by **exactly one paise**: expected 67, charged 66 .

The generator truncated; the calculator rounds HALF_UP.

Had those been filed against a real PSP, **every single one would have been wrong** — and the product's entire premise, "*our arithmetic is right and yours isn't*", would have died on first contact with a customer.

We predicted this in writing four sessions earlier, in OPEN-QUESTIONS.md: "*systematic rounding differences will surface as false breaks.*" That was a hypothesis. 8,174 is a measurement.

It is now a **business control**: `reconpilot.claims.filing-enabled` defaults to `false` , and the error message a user sees explains exactly why.

8.2 Security theatre — RLS enabled and inert

Covered in §3.8. The lesson: **a security control that is enabled but not enforced is worse than none**, because it creates false confidence. I wrote the ADR claiming defence in depth, and believed it until I tested it.

8.3 The test suite that destroyed the dev database

Covered in §5.3. **A test suite that can reach your development database will eventually destroy it.**

8.4 Adding a bean removed another one

Covered in §4.3. `@ConditionalOnMissingBean` means declaring an admin `JdbcTemplate` silently deleted the normal one, routing every query through the privileged connection.

8.5 A rejected upload poisoned its own retry

TWO CORRECT FEATURES, BROKEN BY THEIR COMBINATION

```
POST part05.csv → 503 INGESTION_BUSY ← honest, correct
POST part05.csv → 200 alreadySeen: true ← the file can NEVER be ingested
```

The batch row was written *before* the work was queued, so a rejection left it stranded in RECEIVED. Idempotency keys on the content hash, so that stranded row made every retry report "already ingested".

An honest rejection turned into silent, permanent data loss. Idempotency is correct. Rejecting when full is correct. Together they destroy data.

When you refuse a request, leave the world exactly as you found it. A refusal that changes state is not a refusal.

8.6 The check-then-act race

Covered in §7.9. Invisible over 2,000,000 sequential rows; failed 12 of 14 batches on the first concurrent run.

8.7 Tests that could never run

Covered in §5.1. `*IT` classes compiled and were silently skipped by Surefire.

8.8 The constraint that contradicted the code

Covered in §3.6. We changed the idempotency *query* to allow retrying FAILED batches, and left the *constraint* alone. The code decided to retry; the database refused.

The same rule written in two places will eventually be changed in one.

8.9 The IDE and the build using different JDKs

Covered in §1.2. `JAVA_HOME` in `~/.zprofile` is invisible to `/bin/sh`. Lombok — which we were not even using — crashed on JDK 27. Removing an unused dependency eliminated a whole class of failure rather than pinning around it.

8.10 The API designed without a consumer

FOUND WITHIN MINUTES OF WRITING THE FIRST SCREEN

`GET /api/breaks` returned raw `Map<String, Object>` — leaking the database's `snake_case` naming into the API — and **omitted the break's own id**, so a client could display a finding but could not act on it.

An API designed without a consumer is one that technically works and is unusable.

The pattern across all ten

NOT ONE OF THESE WOULD HAVE BEEN FOUND BY READING THE CODE

Every single one was found by **running something and checking a belief**: firing 16 concurrent uploads, querying the database as a different role, planting known errors, putting a bug back to watch the test fail.

That is worth saying in an interview more plainly than any feature: *"I stress-tested my own system and found these; here is what each one taught me."*

9 • Rebuild it from nothing

Every step, in order, with the reasoning compressed. On a clean Mac this is about a day's work if you already understand the concepts in chapters 1–8.

Step 1 — The machine

```
# Homebrew (the one command that needs your password)
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

brew install openjdk@21 maven node@24
brew install colima docker docker-compose
brew install --cask intellij-idea-ce

# Compose must be registered as a docker CLI plugin
mkdir -p ~/.docker/cli-plugins
ln -sf /opt/homebrew/opt/docker-compose/bin/docker-compose ~/.docker/cli-plugins/docker-compose
```

Then wire the shell — `~/.zprofile`:

```
eval "$(/opt/homebrew/bin/brew shellenv zsh)"

export JAVA_HOME="/opt/homebrew/opt/openjdk@21/libexec/openjdk.jdk/Contents/Home"
export PATH="$JAVA_HOME/bin:$PATH"
export PATH="/opt/homebrew/opt/node@24/bin:$PATH"

# Testcontainers cannot find Colima's socket without these
export DOCKER_HOST="unix://$HOME/.colima/default/docker.sock"
export TESTCONTAINERS_DOCKER_SOCKET_OVERRIDE=/var/run/docker.sock
```

```
colima start --cpu 4 --memory 6 --disk 60
java -version && mvn -version && node -v && docker ps
```

TWO TRAPS AT THIS STAGE

1. `brew install maven` drags in the newest JDK as a dependency. `JAVA_HOME` is what decides which one Maven uses — verify with `mvn -version`, which prints the runtime it chose.
2. IntelliJ ignores `JAVA_HOME` and auto-detects. Set the Project SDK explicitly: **File** → **Project Structure** → **Project** → **SDK** → **21**, and the same under **Settings** → **Build Tools** → **Maven** → **Importing**.

Step 2 — The stack

`~/projects/reconpilot/docker-compose.yml` exactly as in §2.4. Then:

```
cd ~/projects/reconpilot
docker compose up -d
docker compose ps          # both should read (healthy)
```

Step 3 — Design before code

This is the step people skip, and it is the one that saved the most time.

1. **Requirements**, split into functional and non-functional. Rank two of the non-functionals LOW on purpose.
2. **Scale estimation**, out loud, with arithmetic. Ours said *0.4 writes/second* — which decided the entire architecture: one Postgres, a modular monolith, no Kafka.
3. **Read the primary source**. We designed a mid-month threshold engine from accurate news summaries. The regulator's own FAQ showed the mechanism was an account category with a three-month qualifying period. Had we coded first, we would have built the wrong thing and found out during customer disputes.
4. **Write ADRs**. One file per decision: context, decision, consequences — including what you gave up.

TURN THE REGULATION INTO A DECISION TABLE, THEN INTO TESTS

Prose is ambiguous; tests are not. Ours cites the FAQ question each figure came from, so a disputed calculation is defended by pointing at the regulator's own document rather than at our code.

Step 4 — The schema

Write `v1__initial_schema.sql`. The non-negotiables:

- All money `BIGINT` `paise`, with `CHECK (x >= 0)`.
- `occurred_at` **and** `recorded_at` on the event table.
- `raw JSONB NOT NULL` — keep the source row verbatim.
- Append-only: no `UPDATE` or `DELETE` on the event log, ever.
- `UNIQUE (tenant_id, content_hash)` on batches for idempotency.
- `tenant_id` on every tenant-scoped table.

Leave RLS until Step 10 — it needs authentication to be meaningful, and enabling it early breaks every test for no benefit.

Step 5 — Generate the Spring Boot project

```
curl -fsSL https://start.spring.io/starter.zip \
  -d type=maven-project -d language=java -d javaVersion=21 \
  -d groupId=in.reconpilot -d artifactId=backend -d packageName=in.reconpilot \
  -d dependencies=web,data-jpa,postgresql,validation,flyway,actuator,testcontainers \
  -o /tmp/backend.zip
unzip -q /tmp/backend.zip -d ~/projects/reconpilot/backend
```

THREE THINGS TO FIX IMMEDIATELY IN THE GENERATED PROJECT

1. **Do not tick Lombok** unless you will use it. It hooks into private compiler internals and breaks on each new JDK. We removed it and a whole class of failure disappeared.
2. **Rename `BackendApplicationTests` to `ApplicationContextIT`**. It starts a database container, so it is not a unit test.
3. **Pin the Testcontainers image** to the same version Compose runs. The default is `postgres:latest`.

Move the schema to `src/main/resources/db/migration/V1__initial_schema.sql` and set:

```
spring.flyway.enabled=true
spring.jpa.hibernate.ddl-auto=validate
```

Step 6 — The rules engine, test-first

Write the tests from the regulator's published figures **before** the implementation. Then:

- Enums for every dimension (`TxnType` , `PaymentRail` , `PayeeCategory`).
- Records for input and output — and put the **rule id** in the output, not just the amount.
- One ordered evaluation method. Precedence is the contract.
- Unspecified cases **throw**. Never guess a rate.
- The rounding mode is a single named constant with a comment explaining it is provisional.
- **No Spring annotations in this class**. Register it as a `@Bean` in a config class.

```
./mvnw test      # should be milliseconds, no Docker
```

Step 7 — Expose it, then add a UI for yourself

A thin `@RestController` that converts query parameters, delegates, and returns the record. Then a `@RestControllerAdvice` mapping your domain exceptions to honest status codes (501 for unspecified, 400 for bad input).

Add a root endpoint serving a small HTML index of worked examples. It costs twenty minutes and makes every subsequent change verifiable by clicking rather than by assembling URLs.

Step 8 — Ingestion

1. **Prove the memory problem first.** Generate a 150 MB CSV, read it both ways under `-Xmx128m`, and watch one of them die. You will design differently afterwards.
2. Parser returns a lazy `Stream`; hash the file through an 8 KB buffer. Never load it whole.
3. Write with `JdbcTemplate.batchUpdate`, 1,000 rows per flush. Add `?rewriteBatchedInserts=true` to the JDBC URL — free 28%.
4. **One `recorded_at` per file**, computed once.
5. Resolve merchants with `INSERT ... ON CONFLICT ... RETURNING id`, cached in a `HashMap`. Do not check-then-act.

Step 9 — Make it asynchronous

- Synchronous part: validate header, hash, check idempotency, insert the batch row, return **202** with a `Location` header.
- Asynchronous part: a **separate bean** annotated `@Async` — self-invocation bypasses the proxy.
- Bound the pool: 2 core, 4 max, queue 10, `AbortPolicy`. Map the rejection to **503**.
- **On rejection, delete the batch row.** Otherwise the stranded row poisons every retry.
- Re-establish the tenant inside the worker — a `ThreadLocal` does not follow work onto a pool thread.
- A startup sweep marks batches left in RECEIVED/PARSING as FAILED, using the **admin** connection.

Then **fire 16 concurrent uploads**. That is how we found two bugs in ten seconds.

Step 10 — Reconciliation, and validating it honestly

- Walk the batch in 5,000-row **keyset** pages (`WHERE id > ? ORDER BY id LIMIT n`), never OFFSET.
- Index `(batch_id, id)` to support it.
- Classification precedence: exempt, then cap breach, then overcharge, then undercharge.
- Unique index on `recon_break(transaction_event_id)` plus `ON CONFLICT DO NOTHING`, so re-running cannot duplicate findings.
- Unspecified rules are **counted and skipped**, never broken.

VALIDATE WITH PLANTED ERRORS AND AN INDEPENDENT IMPLEMENTATION

Generate the test file with a known number of deliberately corrupted rows, and compute the expected MDR with a **second implementation you write by hand** — not by calling the code under test. Ours disagreed by one paise and exposed 8,174 false breaks.

Step 11 — The dispute workflow

- Transitions as **data** in the enum, with a completeness test over `@EnumSource`.
- **One** transition endpoint, not five.
- Append-only `dispute_event` table recording actor, note and timestamp.

- Guards that encode business rules: an undercharge cannot be claimed; one claim per break; recovery cannot exceed the claim.
- 409 for an illegal transition, 422 for a refused claim.

Step 12 — Authentication and real tenant isolation

Do this before building further on top, not after.

```
-- 1. a least-privilege role: neither owner nor superuser
CREATE ROLE reconpilot_app LOGIN PASSWORD '...';
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO reconpilot_app;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO reconpilot_app;

-- 2. force RLS so it applies to the owner too
ALTER TABLE transaction_event FORCE ROW LEVEL SECURITY;

-- 3. policies governing reads AND writes
CREATE POLICY tenant_isolation ON transaction_event
    USING      (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::uuid)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::uuid);
```

On the Java side:

1. **Two datasources.** App as `reconpilot_app`; Flyway and system tasks as the owner.
2. **Declare the primary `JdbcTemplate` explicitly** and mark it `@Primary` — otherwise declaring an admin one silently deletes it.
3. **Honour `JdbcConnectionDetails`** in your `DataSource` bean, or you disconnect Testcontainers and your tests will point at your dev database.
4. A `TenantAwareDataSource` that sets the tenant on **every checkout** and clears it on **close**.
5. JWT filter establishing the tenant from the *signed token*, clearing the `ThreadLocal` in a `finally`.
6. BCrypt for passwords. Same 401 for unknown email and wrong password. 401 entry point, not 403.

MAKE THE TEST SUITE PROVE IT IS TALKING TO A CONTAINER

```
if (!url.equals(postgres.getJdbcUrl()))
    throw new IllegalStateException("Refusing to run: not the test container");
```

Add this before you write a single `TRUNCATE`. We added it afterwards, having lost a million rows.

Then write the isolation test that **connects as the app role** — a test running as the owner would pass while proving the opposite of what it claims.

Step 13 — The frontend

```
npm create vite@latest frontend -- --template react-ts
cd frontend && npm install
npm install @mui/material @emotion/react @emotion/styled @mui/icons-material \
  @reduxjs/toolkit react-redux framer-motion react-router-dom recharts
```

- **One** design system. Not two.
- Vite proxy for `/api` — sidestep CORS rather than configuring around it.
- RTK Query with `tagTypes` and `invalidatesTags`.
- `prepareHeaders` attaches the token in one place; a base-query wrapper signs out on any 401.
- Server state in RTK Query, client state in a small `uiSlice`. Keep them apart.
- Mirror the state machine to decide which buttons to *offer* — the server remains the authority.

EXPECT THE FIRST SCREEN TO BREAK YOUR API

Ours exposed two defects within minutes: raw maps leaking `snake_case`, and a missing id that made rows undisplayable-but-unactionable. Design the API *with* its first consumer.

Step 14 — Wire up the build properly

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-failsafe-plugin</artifactId>
  <executions><execution><goals>
    <goal>integration-test</goal><goal>verify</goal>
  </goals></execution></executions>
</plugin>
```

```
./mvnw test      # *Test  - fast, no Docker
./mvnw verify    # + *IT   - the full check
```

And for every regression test you write: **put the bug back and watch it fail**, then restore the fix.

10 • Interview answers

The short versions. Each one is backed by something you measured.

"Tell me about a project."

I built a reconciliation platform for India's new UPI MDR rules. It ingests settlement files — two gigabytes, millions of rows — independently recomputes what each merchant should have been charged under the NPCI framework, and drives the differences to recovery. On a million-row file it finds around 7,300 discrepancies worth ₹1.3 lakh in three seconds, in a 256 MB heap.

"How did you decide the architecture?"

I estimated first. A thousand merchants at a thousand transactions each is 0.4 writes per second average, 280 at peak, and about 5.5 GB a year. A single Postgres instance handles that for years. So this is not a scale problem, it is a correctness problem — and I spent the complexity budget on determinism and auditability rather than on distribution.

"Why not microservices?"

Because nothing in the numbers asked for them. They would have added network partitions, partial failures and eventual consistency to a system whose hard requirement is paisa-exact correctness. I built a modular monolith with enforced boundaries, so any module could be extracted later if a real reason appeared.

"How do you handle a 2 GB file?"

Stream it. I measured both ways: reading a 148 MB file whole used 251 MB of a 256 MB heap and died below that, because each line becomes a String object costing about $2.7\times$ the file size. Streaming did the same work in 23 MB. Memory has to be independent of input size — the dangerous case is the one that almost fits, because it ships and then dies at 3 a.m. on your biggest customer's biggest file.

"How do you guarantee you don't double-process a file?"

A SHA-256 of the file bytes under a unique constraint, so Postgres rejects it rather than application code that someone can forget. A duplicate 148 MB upload returns in 652 milliseconds having written nothing. The same pattern appears three times in the system.

"Tell me about a bug you found."

I enabled row-level security for multi-tenant isolation, then tested whether it actually worked — and it did nothing. Postgres exempts a table's owner from RLS unless you also FORCE it, and exempts superusers entirely, and the app connected as both. Two million rows were visible with no tenant context. I'd written an ADR claiming defence in depth and believed it. The fix needed a least-privilege role, FORCE, and WITH CHECK on the policies — USING alone governs reads, so a tenant could otherwise write rows it couldn't see.

"Tell me about a concurrency bug."

Merchant creation was a check-then-act: SELECT, then INSERT if absent. Two million rows sequentially never hit it. The first burst of 16 concurrent uploads failed 12 of 14 batches with a duplicate key. I replaced it with an atomic INSERT ... ON CONFLICT ... RETURNING, letting Postgres resolve the race. The lesson is that any check-then-act has a gap, and something will eventually land in it.

"How do you know your tests are any good?"

I put the bugs back and watched the tests fail, then restored the fixes. A regression test that has never failed is a hypothesis, not a test. I also validated the detection engine by planting a known number of errors — 7,300 — and requiring it to find exactly those, with the expected values computed by a second implementation I wrote by hand. That caught a one-paise rounding disagreement that was producing 8,174 false claims.

"What's the hardest part of the domain?"

Rounding. The regulator publishes 0.4% but never states a rounding rule. One paise of disagreement between our arithmetic and a PSP's produces thousands of false claims — and the product's whole premise is that our arithmetic is right. So filing is disabled by default in code, and the error message explains why. That's an engineering measurement turned into a business control.

"What would you do differently or next?"

Three things. Generate the TypeScript types from an OpenAPI spec instead of mirroring them by hand. Move the ingestion queue outside the process — not for throughput, which we don't need, but so work survives a restart. And add the AI layer for normalising PSP statement formats, which is the part that doesn't generalise well in code.

THE SENTENCE TO END ON

Most candidates describe features they built. The stronger move is to describe **beliefs you tested and found false** — the security control that did nothing, the test suite that destroyed a database, the rounding difference that would have destroyed customer trust. That is what senior engineering actually looks like, and very few people at two years' experience can show it.